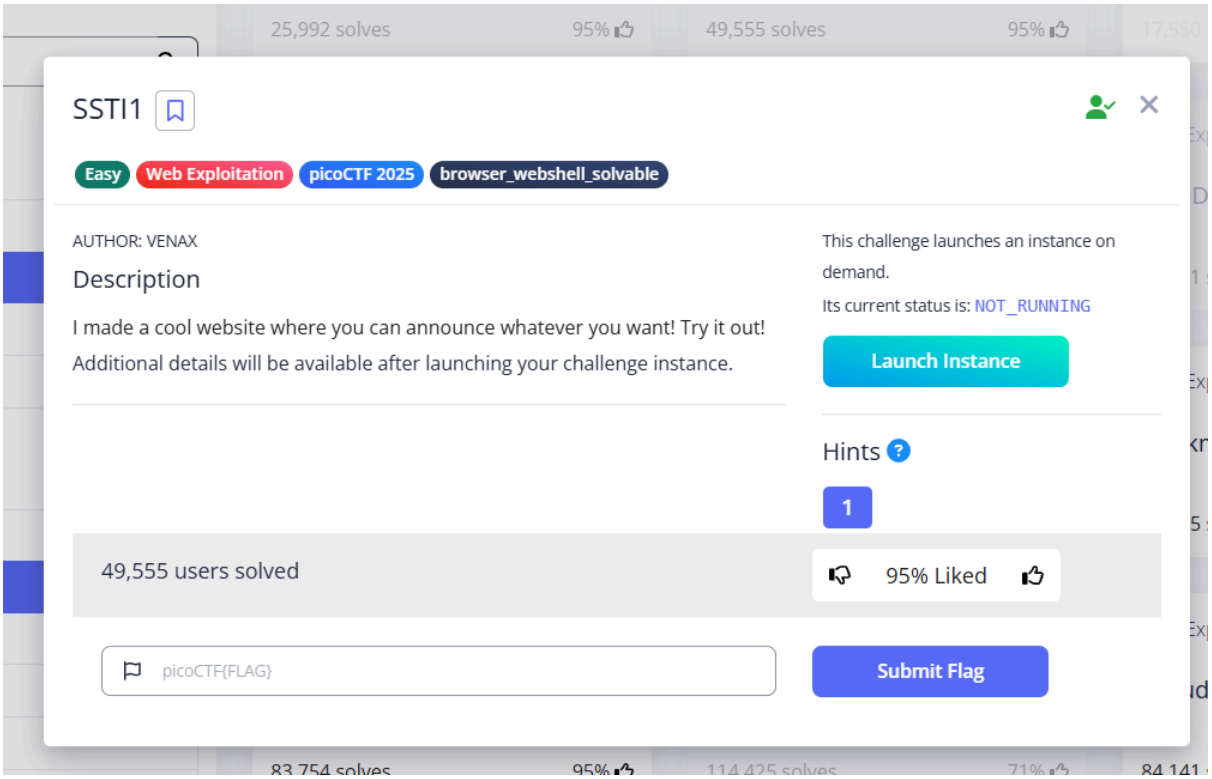
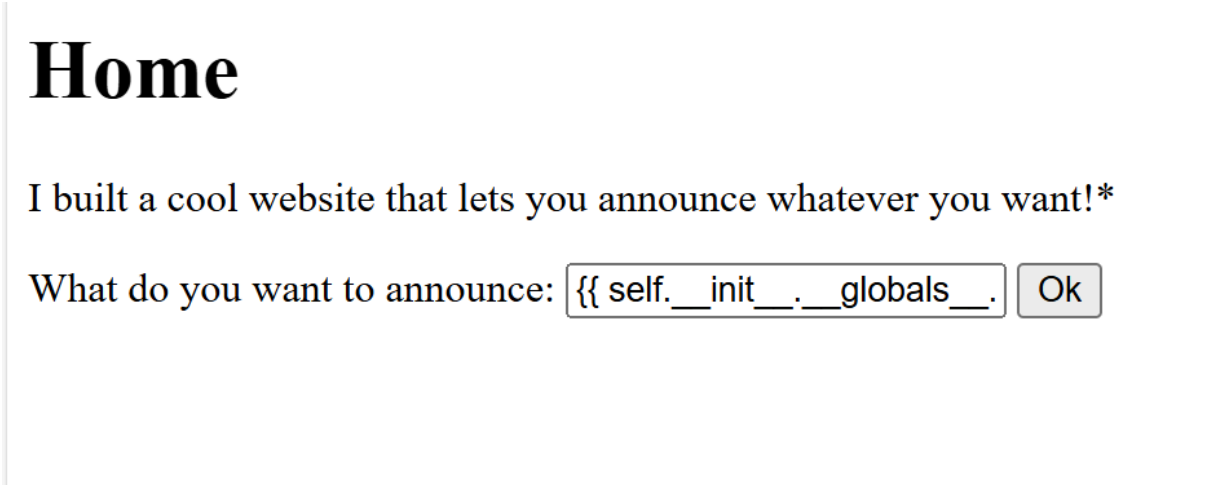
JASMINE C. OMANDAM

picoCTF - picoGym Challenges





picoCTF{s4rv3r_s1d3_t3mp14t3_1nj3ct10n5_4r3_c001_f5438664}

Write - Up: SSTI1

When I first encountered the challenge, the website looked deceptively simple. A single form asked me to type an announcement, and whatever I submitted was reflected back on the page. At first glance, it seemed harmless no hidden parameters, no complex scripts, just a straightforward echo of user input. But simplicity often hides danger, and this was one of those cases.

Step 1:

The HTML source revealed nothing unusual. A basic form with one input field, no JavaScript trickery, no suspicious endpoints. The site claimed that announcements "may only reach yourself," which hinted at isolation. Yet, the fact that user input was directly displayed raised a red flag. Whenever data is echoed back, the question becomes: is it being safely handled, or is it being processed by something more powerful?

Step 2:

Classic vulnerabilities came to mind:

- XSS could be possible if the input wasn't sanitized, but that would only affect the browser.
- SQL Injection didn't fit, since there was no evidence of database interaction.
- Server-Side Template Injection (SSTI) seemed the most likely, especially given the challenge's name. SSTI occurs when user input is passed into a template engine and executed as code.

Step 3:

To confirm SSTI, I tested with a simple payload:

```
{{ 7*7 }}
```

If the site was using Jinja2, this would evaluate to 49. Sure enough, the output wasn't the raw string—it was the computed result. That was the confirmation I needed: the site was vulnerable to Jinja2 SSTI.

Step 4:

Once SSTI was established, the next step was to explore what objects were accessible. Jinja2 templates often expose Python internals. By injecting:

```
{{ self.__init__.__globals__ }}
```

I uncovered a dictionary of global variables, including `__builtins__`. This was critical, because `__builtins__` provides access to Python's core functions, opening the door to importing modules and executing system commands.

Step 5:

With access to `__import__`, I could pull in Python's `os` module and run commands directly on the server. For example:

```
{{
self.__init__.__globals__.__builtins__.__import__('os').popen(
'ls').read() }}
```

This listed the server's directory contents. From there, retrieving the flag was trivial:

```
{{
self.__init__.__globals__.__builtins__.__import__('os').popen(
'cat flag').read() }}
```

The server obediently returned the flag, proving full code execution was possible.

picoCTF{s4rv3r_s1d3_t3mp14t3_1nj3ct10n5_4r3_c001_f5438664}

This challenge highlights how dangerous SSTI can be. What looked like a harmless announcement form turned into a gateway to arbitrary code execution. The lesson is clear: never pass user input directly into a template without escaping it. Proper sanitization and separation of logic from presentation are essential to prevent such attacks.

.